

Tecnologie Web: Appunti HTML

Contents

Fonti e siti utili	3
HTML5	3
Sintassi XHTML	4
Alcune semplificazioni fatte da HTML	4
Prologo XML	4
Elementi che vengono ignorati dai browser	4
Come sono formati i file HTML	4
Intestazione	4
Elementi dell'Head	5
Corpo del documento	5
Elementi del body	5
Markup strutturale	6
Citazioni	7
Abbreviazioni, acronimi, indirizzi	7
Tag per testo particolare	7
Elenchi	8
Immagini	8
Lazy loading	8
Link	9
Accesso ai link da tastiera	9
Tabelle	9
Regole per le tabelle	10
Raggruppare le righe	10
I form	11
Formato della query string	11
Campi e pulsanti di una form	11
Il tag <code><input></code>	11
Checkbox e radio button	12
Hidden	12
File	12
Tag <code><textarea></code>	12
Tag <code><select></code>	12
Tag <code><datalist></code>	12
Innovazioni	12
Innovazioni per le form	12
Canvas	13
Audio	13
Video	14

Lavorare in locale	14
Cache manifest	14

Fonti e siti utili

- Specifiche W3C per XHTML
 - <http://www.w3.org/TR/html5/>
- MDN
 - <https://developer.mozilla.org/en-US/>
- Validatore sintassi XHTML
 - <http://validator.w3.org/>
- Can I Use
 - <https://caniuse.com/>
 - Sito che per ogni browser dice in quale versione è compatibile con certe funzionalità
- Esempi XHTML
 - sito che per ogni browser dice in quale versione è compatibile con certe funzionalità
 - <http://www.htmldog.com/examples/>
- Tutorial XHTML
 - <http://www.htmldog.com/>
 - <http://www.w3schools.com/xhtml/default.asp>
 - <http://xhtml.html.it/guide/leggi/52/guida-xhtml/>
- Specifiche HTML5
 - <http://www.w3.org/TR/html5/>
- Tutorial HTML5
 - <http://www.w3schools.com/html5/default.asp>
 - (Tutorial sui Canvas) https://developer.mozilla.org/en/Canvas_Tutorial
 - (Tutorial e esempi) <http://www.html5rocks.com/en/>
 - <http://www.html5rocks.com/en/mobile/mobifying/>
- Altri siti di riferimento
 - <http://html5doctor.com/>
 - “Dive into HTML5” Mark Pilgrim
 - * <http://diveintohtml5.info/>
- +50% degli accessi ai siti avvengono da telefono
- Dobbiamo produrre un html che sia valido. Nella validazione si controlla che il codice sia sintatticamente corretto (!= compilazione)
 - Il validatore è nei link utili
- Accessibilità e ranking sono il motivo per cui sono state fatte la maggior parte delle scelte

HTML5

- Per progetto: HTML con sintassi XML
- Tre elementi da separare: Comportamento (JavaScript), Struttura (codice html) e Presentazione (css)
 - La divisione rende tutto più mantenibile

Sintassi XHTML

- i tag e gli attributi sono case sensitive (tutto in minuscolo)
- i tag devono sempre essere chiusi (anche se sono vuoti)
 - `<br />` e non `<br>`
 - in realtà nei tag su singola riga si può anche non mettere la chiusura con `/`
- i tag devono essere aperti e chiusi nell'ordine corretto
- l'ordine con cui si inseriscono gli attributi è irrilevante
- i valori degli attributi vanno riportati tra virgolette doppie `" "`
- tutti gli attributi devono avere un valore
 - può essere ignorata solo per quegli attributi in cui il valore è il nome dell'attributo
- un elemento in linea non può contenere un elemento di blocco
- Se i browser trovano del codice non valido cercano di visualizzarlo al meglio, ma questo può portare ad interpretazioni arbitrarie

Alcune semplificazioni fatte da HTML

- `<!DOCTYPE html>`
- `<meta charset="utf-8">`
- `<script rel="script.js"></script>`
- `<link rel="stylesheet" href="print.css"/>`

Prologo XML

- Il W3C raccomanda di usare il prologo XML ma molti browser non lo gestiscono bene e causano errori, quindi è **da evitare**.
- `<?xml version="1.0" encoding="ISO-8859-1"?>`

Elementi che vengono ignorati dai browser

- le interruzioni di linea non identificate con `<br/>` e non contenute in un tag `<pre>`
- tabulazioni e spazi multipli
- tag `<p>` nidificati
 - `<p>` nidificati sono considerati un ERRORE
- tag sconosciuti
- commenti:
 - ATTENZIONE: all'interno di un commento non è possibile inserire la stringa `"–"` (doppi trattini)

Come sono formati i file HTML

- `<!DOCTYPE html>` è la prima riga, dice al browser che tipo di file ha davanti
- `<html lang="it">` definisce la lingua principale [Accessibilità]
- `<meta charset="UTF-8" />` dice che tipo di caratteri sto usando. Questo tag meta deve essere contenuto nei primi 512 bit, altrimenti il browser decide da solo.

Intestazione

- La parte contenuta tra i tag `<head>` e `</head>` viene chiamata intestazione, o semplicemente sezione head.
- In questa sezione si trovano tutti i tag che impartiscono direttive al browser quali: titolo (obbligatorio), comandi meta, richiami ai fogli di stile, script.
- Tutto ciò che si trova all'interno della struttura head non sarà visualizzato ma verrà interpretato dal browser (tranne il titolo, quello lo vede anche l'utente).

La sezione head quindi è una zona destinata ad uso esclusivo dei soli comandi che impartiscono direttive ben specifiche.

Elementi dell'Head

- **title**(obbligatorio), **link**,**meta**,**base**,**style** e **script**
- I **link** sono collegamenti a risorse esterne (CSS, icon, ecc)
 - I suoi attributi più comuni sono **href** e **rel**(relazione). Es:
 - * `<link rel="stylesheet" href="/file.css" />`
 - * `<link rel="shortcut icon" href="/img.ico" />`
 - * `<link rel="next" title="Next page" href="/next.html" />`
 - da evitare quest'ultima
- La **base** definisce la posizione di base per i collegamenti
 - `<base href="/miacartella/" />`
- **style** e **script** servono per inserire codice css e js direttamente nella pagina HTML, non sono quindi da usare perché dobbiamo rispettare il principio di separazione. Questi tag sono presenti solo per retrocompatibilità
- I tag `<meta http-equiv="">` possono condizionare il funzionamento della pagina
- Tag `<meta name="">` , danno info riguardo al documento. È possibile creare dei propri valori. Opzioni disponibili:
 - **description** breve descrizione della pagina. È importante che contenga le keywords
 - **keywords**: lista di parole chiavi separate da una virgola. Se le pagine contengono effettivamente le keyword aumenta la trustness. Meglio che siano presenti anche nei titoli oltre che nei paragrafi.
 - **copyright**
 - **author**
 - **robots**
 - **rating**
 - **viweport** aiuta ad adattare i siti su dispositivi diversi (es telefoni)

Corpo del documento

- La parte contenuta tra i tag `<body>` e `</body>` viene chiamata corpo del documento o semplicemente, sezione body.
- L'obiettivo è dare una struttura semantica, ovvero dividere il testo in parti e indicare l'importanza delle parti.
- La sezione body contiene quindi tutti i tag che descrivono la struttura del documento: non devono essere usati elementi relativi alla presentazione visuale
- Si devono usare gli elementi per il loro significato e non per come vengono visualizzati dai browser

Elementi del body

- Attributi comuni, possono essere messi a qualsiasi tag
 - **class**, assegna una classe di riferimento a cui possono appartenere diversi elementi, anche tag diversi
 - **id**, assegna un identificativo unico all'interno della pagina. Ha maggiore specificità della classe quando si fa il css. Può essere usato come destinazione di un id. Ha maggiore supporto javascript anche con browser vecchi
 - **title**, aggiunge un titolo ad un elemento
 - **style**, istruzioni css in linea per quel tag

- `xml:lang` indica la lingua. Può essere usato per specificare la lingua di un certo paragrafo se questa non è la stessa della pagina
- attributi evento, per attaccarci eventi javascript
- Tag generalisti, non hanno nessun valore semantico. Se ciò che si vuole scrivere ha un valore semantico serve usare tag diversi
 - `<div>...</div>` contenitore generico, serve per poter associare il suo contenuto a un foglio di stile. Gli attributi associati a un `<div>` si diffondono su tutto il suo contenuto.
 - `<span>...</span>` come il `<div>` ma su una sola linea
 - * es. di uso: quando c'è una parola in un'altra lingua, si mette quella parola tra `<span>` e si dà l'attributo `lang` indicando la lingua.
 - A livello visivo non cambia niente ma il motore di ricerca/screen reader sanno che quella è una parola in un'altra lingua.

Markup strutturale

HTML5 aggiunge dei tag per descrivere meglio la struttura del documento:

- `<header>` e `<footer>`, rispettivamente intestazione(o banner) e piè di pagina. Possono essere presenti più volte nella stessa pagina/sezione. Il `<footer>` identifica le info su chi ha scritto i contenuti.
- `<main>`, contenuto principale
- `<nav>`, elementi di supporto alla navigazione. Può essere contenuto nell'header
- `<aside>`, parte del contenuto che può essere rimossa senza diminuire il significato della pagina, fa da supporto
- `<section>`, per raggruppare contenuti sullo stesso tema o logicamente collegati
- `<article>`, porzione di testo autocontenuto e indipendente dal resto del documento che possa essere distribuito in modo autonomo
- Non ricorrere a `<section>` e `<article>` per soli motivi di stile. Usare il `<div>` in quel caso.
- `<article>`, `<nav>`, `<section>` e `<aside>` sono sectioning element, ovvero possono contenere `<header>`, `<nav>` e `<footer>`

Per controllare se il documento è stato strutturato bene una buona possibilità è verificare il sommario generato automaticamente. A tale scopo su Chrome è disponibile l'estensione Silktide.

Se il browser non supporta il markup strutturale esiste una libreria Javascript che la interpreta per lui.

Esempio header (corretto):

```
<header>
  
  <div id="headerText">
    <nav>
      <form>...</form>
    </nav>
    <h1>Corsi di laurea in informatica</h1>
    <p>Sito dedicato ai corsi di laurea in informatica</p>
  </div>
```

In diversi siti nel tag `<header>` si potrebbe trovare scritto `<header role="banner">`. `<header>` definisce di base il banner, `role="banner"` si usava col `<div>` per dire che quel `<div>` aveva il ruolo di banner. Adesso non è necessario metterlo perché è già considerato "di default".

- Caratteri speciali
 - " si inserisce con `"`

- < si inserisce con `<`
- > si inserisce con `>`
- € si inserisce con `&euro`
- & si inserisce con `&`
- Enfasi del testo
 - `<em>` `</em>` = enfasi, sostituisce `<i>` (italic)
 - `<strong>` `</strong>` = forte enfasi, sostituisce `<b>` (bold)
- Intestazioni(heading)
 - sei livelli di intestazione: h1,h2 ... h6
 - va rispettato l'ordine, non puoi saltare da h1 ad h5
 - bisogna pensare alla struttura del documento e non a come vengono visualizzati
 - * la visualizzazione può essere modificata con css
 - **IMPORTANTE:** Non mettere troppi heading, o heading troppo lunghi, nel sito perché il browser pensa che lo fai per ingannarlo per dare più valore alle tue keyword.

Regole per scrivere i titoli: - Scrivi un titolo unico per ogni pagina - Cerca di essere conciso e descrittivo - Evita titoli vaghi e generici - Utilizza la maiuscola per la prima lettera della frase o la prima lettera di ogni parola - Crea contenuti degni di click ed evita i clickbait - Pensa all'intento di ricerca - Includi la parola chiave principale quando ha senso farlo - Massimo 60 caratteri

La marcatura del testo deve corrispondere al significato semantico dell'elemento in essa racchiuso:

```
<body>
  <p class="titolo">....</p>
  <p>...</p>

  <p class="sottotitolo">....</p>
  <p>.. </p>
</body>
```

Citazioni

- `<blockquote>` è un tag di blocco che riporta una citazione.
Il tag `<q>` è il corrispondente di `<span>` (quidi su singola riga) con la caratteristica semantica di una citazione.
- si può usare l'attributo `cite` per dare una fonte, necessita obbligatoriamente di un url, in alternativa c'è `title`
- in HTML5 il tag `<cite>` esiste ma deve essere il titolo di un libro, film ecc.

Abbreviazioni, acronimi, indirizzi

- acronimo è diverso da abbreviazione. Acronimo è quando leggo tutte le lettere (es. NATO), abbreviazione è quando accorcio una parola (come quando scrivo “es.” invece di “esempio”)
 - nel caso delle abbreviazioni devo indicare nell'attributo `title` per esteso cosa vuol dire

Tag per testo particolare

- `<code>`, per il codice
- `<var>`, indentifica delle variabili nel codice
- `<samp>`, per quando c'è l'outup di un programma
- `<pre>` testo preformattato, dove spazi e a capo hanno valore
- `<ins>` : identifica un inserimento redazionale. Solitamente è visualizzato sottolineato
- `<del>` : identifica una cancellazione redazionale. Solitamente è visualizzata barrata

- Possono essere usati sia come elementi in linea che di blocco

Alcuni tag semantici più interessanti sono - **<time>**, con attributo **datetime** che contiene data/ora in formato XML - **<mark>** evidenzia il testo - (tralasciando gli heading) i motori di ricerca danno la seguente importanza: **strong > em > mark** - **<meter>** indica una scala con un max e un min - **<progress>**, per indicare un valore che sta cambiando. Va usato con javascript se si vuole far vedere una barra che avanza.

Se un valore che voglio mostrare è fisso non uso **progress** per indicarlo. - **<small>** note a piè pagina - **<figure>/<picture>** - se voglio solo inserire un'immagine uso **** - se voglio aggiungere un testo all'immagine (come una didascalia all'immagine nei libri) allora uso **<figure>** - se voglio inserire un'immagine che cambia (risoluzione o formato) in base al dispositivo, uso **<picture>**

Elenchi

Keyword che appaiono all'interno di un punto elenco sono considerate di più dal browser rispetto a quelle che appaiono nel testo normale.

Gli elenchi sono molto importanti perché attirano l'attenzione. Un testo organizzato per punti elenco è più facilmente leggibile.

Ci sono tre tipi di elenchi: - ****, unordered list - i cui elementi vanno identificati con **** list item - **** ordered list - anche qui gli elementi sono **** - l'attributo **reverse** permette di invertire la lista - l'attributo **start** dice da che numero partire - l'attributo **type** dice il tipo di marcatore. **DA NON USARE**, se serve cambiare marcatore lo si fa da css - si può attribuire un **value** ai punti della lista

- **<dl>** liste di definizione, come in un enciclopedia, un elenco di voci che vengono poi descritte
 - **<dt>** indica la voce da definire
 - **<dd>** indica la definizione della voce, possono esserci più **<dd>** per ogni **<dt>**

La navigazione dovrebbe contenere il menù come elenco di link. I link attivi non devono avere la **<a>** perché i browser ad oggi mettono le pagine caricate in cache e se ci si preme non appare neanche il caricamento della pagina e all'utente sembra che il sito non funzioni. **ERRORE GRAVE**

Immagini

Il tag principale è ****. Attributi: - **src** indica l'immagine da prendere - **alt** indica il testo alternativo. Testo che descrive l'immagine. Deve essere breve, per aiutare ma non rallentare troppo. Metterlo È **OBBLIGATORIO**, anche se vuoto deve esserci - **longdesc**, dà un URI a una pagina che descrive l'immagine, per immagini che non possono essere descritte velocemente. **DEPRECATO** - **width** e **height**, è possibile definire altezza e larghezza sia in HTML che in css. Nella stragrande maggioranza dei casi si fa nel CSS per mantenere la divisione tra presentazione e contenuto, però metterlo in html PUÒ migliorare l'esperienza utente. Questo perché l'html è il primo file che il browser riceve, se la dimensione dell'immagine è già lì il rendering è più veloce. Se la pagina è particolarmente complessa ha senso mettere le dimensioni all'interno dell'html.

Lazy loading

Raramente una pagina è completamente visualizzata, spesso per vederne il resto usiamo lo scroll. Visto che dobbiamo far caricare il più velocemente possibile le pagine, possiamo dire al browser che un'immagine va caricata solo quando viene visualizzata.

Si fa tramite l'attributo **loading**, che può avere il valore **eager** (default) che carica subito l'immagine, oppure **lazy** che la carica solo quando visibile. Questo velocizza il caricamento iniziale ma potrebbe rallentare il caricamento durante lo scrolling. Per non appesantire la pagina si dà questo attributo solo alle immagini "below the fold" ovvero quelle che non sono visualizzate senza lo scroll.

Per aggiungere una didascalia:

```
<figure>
  
  <figcaption>Didascalia della figura</figcaption>
```


</figure>

Una figura non deve necessariamente avere un'immagine. Un'immagine che è associata a una didascalia non ha bisogno dell'attributo **alt** per rispettare i vincoli di accessibilità. Però potrebbe essere comunque necessario.

Le keyword all'interno di un **alt** sono importanti.

Esempio di uso del <picture>:

```
<picture>
  <source media="(min-width:650px)" srcset="img_pink_flowers.jpg">
  <source media="(min-width:465px)"srcset="img_white_flower.jpg">
  
</picture>
```

Link

Tag <a> per inserire un link. La destinazione sta all'interno dell'attributo **href**. All'interno del tag può esserci anche un'immagine e in quel caso l'**alt** non deve descrivere l'immagine ma il suo scopo.

Per evitare il disorientamento serve che gli utenti riconoscano i link e capiscano dove li porta. La sorgente del link deve essere significativa, sia per l'utente che per il browser. È bene che non sia un'immagine, se lo fosse occorre comunque mettere del testo (**alt** o image replacement) per indicare a cosa porta. Nel caso di un logo che porta alla home, l'**alt** è "home pagina" e non la descrizione del logo.

Se il link non porta a una pagina ma (per es.) a un download serve specificarlo e dire la dimensione del download.

È bene non usare mai path assoluti ma solo relativi. L'attributo **target** indica il frame in cui aprire la finestra (se non inserito, di default apre il link in una nuova finestra). NON si apre una nuova pagina se la destinazione del link è interna al sito, è ok se cambi lingua, se vai su un'altro sito o se apri un file non html (es. pdf).

Link non ipertestuali: che non rimandano ad HTML ma ad altre cose.

Accesso ai link da tastiera

Attributi che aiutano chi non è in grado di usare il mouse ad accedere ai link. - **accesskey**, sono scorciatoie per gli screenreader o browser per aprire il link. Non esistono più tuttavia lettere libere. - **tabindex** definisce l'ordine con cui viene passato il tab. Il tab raggiunge tutto quello che è interagibile, il primo che viene selezionato è quello con **tabindex** più alto (di default è 1). Di base se la struttura del mio sito è corretta non serve mettere valori sopra 1. Quando il **tabindex** è 0 vuol dire che obblighiamo un elemento che non riceverebbe il focus lo obblighiamo a riceverlo. Quando il **tabindex** è -1 si toglie dalla sequenza del focus un oggetto che di base riceverebbe il focus. Lo faccio quando magari voglio nascondere degli elementi.

Il focus deve essere SEMPRE visibile.

Tabelle

Non vanno mai usate a scopo di presentazione (si faceva una volta ma non è corretto). Sono difficili da ridimensionare.

Una tabella si crea con il tag <table>. <tr> e <td> indicano, rispettivamente, le righe e le colonne. Intere tabelle possono poi essere a loro volta contenute in celle di altre tabelle, che vengono quindi nidificate.

```
<table>
  <tr>
    <td> qui andrà messo il contenuto della tabella </td>
  </tr>
</table>
```

Regole per le tabelle

- Non ci possono essere righe senza celle al loro interno
- Le colonne non si definiscono in modo esplicito ma si definiscono le celle all'interno delle righe tramite gli elementi `<td>`
- Ci possono essere celle che occupano più colonne (`colspan`) o più righe (`rowspan`). QUESTA C'È NELLO SCRITTO
- È possibile creare delle intestazioni per le colonne/righe con gli elementi `<th>` al posto di `<td>`
- Il tag `<caption>`, posto subito dopo il tag `<table>`, permette di inserire un titolo (in genere visualizzato sopra la tabella). Le keyword all'interno della `<caption>` pesano di più per il ranking rispetto a paragrafi o `<td>`
- La struttura della tabella in passato veniva descritta nell'attributo `summary` oggi rimpiazzato da un riferimento contenuto in un `aria-describedby`. Qui dentro non ci va il titolo, ma la descrizione della tabella, per far capire com'è organizzata.

Raggruppare le righe

È necessario dividere le celle di intestazione dalle celle di dati (OBBLIGATORIO PER PASSARE VALIDAZIONE).

- `thead` righe di intestazione
- `tbody` righe di dati
- `tfoot` usato in alcuni casi per i totali o per ripetere quello che c'è nel `thead` per aiutare la visualizzazione. Si mette prima del `tbody` così se la tabella è lunga più pagine in automatico li ripete.

Per avere le righe di colori alterni ci sono due modi: - mettere alle righe pari una classe (es. `tr class="alternative"`) a cui poi nel css darò un'altra impostazione. Funziona anche con browser vecchi. - con css3 posso dirlo direttamente. Non è supportato da tutti.

```
<table>
  <thead>
    <tr><th>1 col</th><th>2 col</th><th>3 col</th></tr>
  </thead>
  <tbody>
    <tr><td>Dato 1</td><td>Dato 2</td><td>Dato 3</td></tr>
    <tr class="alternative"><td>Dato 4</td><td>Dato 5</td><td>Dato 6</td></tr>
    ...
  </tbody>
</table>
```

Si può fare anche per le colonne. Sebbene le tabelle si costruiscano per righe, è possibile indirizzare le colonne, per creare effetti di layout ad esse associati. - `colgroup` consente di applicare attributi ad un set di colonne identificato dall'attributo `span` (simile a `rowspan` e `colspan`) - `<col>` permette di selezionare una singola colonna

```
<table>

  <colgroup>
    <col />
    <col class="alternative" />
    <col />
    <col class="alternative" />
  </colgroup>

  <tr>
    <td>This</td>
    <td>That</td>
```

```

        <td>The other</td>
        <td>Lunch</td>
        <td>Lunch</td>
    </tr>
    ...
</table>

```

I form

Permettono di raccogliere dati o input.

```

<form action="http://server/path/file.php" method="post" >
    <!-- elementi del form -->
</form>

```

Due attributi fondamentali: - **action**, chi risponde alla form - **method**, il tipo di richiesta

Domanda che c'è probabilmente nello scritto: differenza tra metodo get e metodo post

Metodo get: è il predefinito, mette tutti i parametri nell'URL. È limitato dalla lunghezza ed è in chiaro(vulnerabile).

Metodo post: viene utilizzato per inviare dati. Se voglio spedire un file non ho altri modi. È molto più sicuro e non ho limiti di lunghezza, però devo compilare la form ogni volta.

Formato della query string

- Contiene i dati inviati cliccando il pulsante Submit
- Il nome e il valore di ciascun elemento della form sono codificati come assegnamenti
 - Ex. nome=Mario&Cognome=Rossi
- I caratteri speciali sono codificati sottoforma di numeri esadecimali preceduti da %
 - Ex. Lo spazio è rappresentato da %20: Nome=Mario%20Rossi
- Con il metodo get la pagina di destinazione può essere salvata come bookmark in modo da poter ripetere la query senza reinserire i dati
- Se si usa il metodo get la stringa viene inserita dal server in una variabile d'ambiente
- Se si usa il metodo post si deve leggere la stringa di query dall'input standard

Campi e pulsanti di una form

- **input**
- **textarea**
- **select**

Hanno attributi particolari - **name**: serve per identificare l'input inviato al server. Ogni elemento viene inviato come una coppia nome-valore. Il nome si ricava da questo attributo, il valore è l'input inserito dall'utente in quel campo. - È DIVERSO DA ID. Id serve per indicare un elemento unico e potervi accedere con css o javascript, name serve per spedire dati al server. Non sempre coincidono - **readonly="readonly"**: i campi con questo attributo non sono editabili dall'utente. - **disabled="disabled"**: i campi con questo attributo non sono editabili dall'utente. Il valore di questo campo non viene inviato al server.

Il tag <input>

Questo tag permette da solo di creare diversi elementi di una form a seconda del contenuto dell'attributo **type**: - **type="text"**: una singola riga di testo con maxlength elementi - **type="password"**: una riga di testo offuscata - **type="checkbox"**: un semplice on/off - **type="radio"**: per selezionare una o più opzioni - **type="submit"**: pulsante per inviare i dati del modulo - **type="reset"**: pulsante per riportare i valori predefiniti nei campi del modulo - **type="hidden"**: per dati non visibili o non editabili - **type="file"**: per caricare file - **type="button"**: per richiamare script lato client - **type="image"**: da non usare

<fieldset>, tag usato in caso di form molto lunghi per raggruppare gli elementi di una form. Disegna un riquadro attorno agli elementi. Quando c'è sto tag è obbligatorio mettere anche la **<legend>** per aggiungere un'intestazione. Va messo subito dopo l'apertura di **<fieldset>**.

Per ogni input serve una **<label>** che indica cosa ci si aspetta in quel input. Non usare mai la **<p>** al posto della **<label>** perché solo con **<label>** ci posso "agganciare" l'input. Nel tag **<label for="">** all'interno del **for** ci va l'id dell'input a cui si riferisce. Non necessariamente **label** e **input** devono essere adiacenti.

Checkbox e radio button

Le linee guida dicono che la dimensione minima che un oggetto deve avere per poter essere interagibile da smartphone è di 44x30 pixel. Se si usano le **<label>** con le **checkbox** basta premere anche sulla **<label>** e non per forza sul quadratino.

radio button permette una sola selezione mentre **checkbox** permette una scelta multipla.

Nel caso della **checkbox** essa viene inviata al server solo se selezionata. Viene inviato o il valore **on** oppure il valore specificato nell'attributo **value**. Nel caso del **radio button**, c'è sempre una e una sola opzione selezionata e serve specificare un valore di default.

Hidden

I tag **<input>** di tipi **hidden** non vengono visualizzati e non possono interagire con l'utente. Possono essere usati per: - passaggio dati in modo da non richiederli all'utente in una sequenza di form - salvataggio di informazioni calcolate sulla base dei dati inseriti dall'utente - definizione di variabili

File

L'input di tipo file serve per fare l'upload di un file dal computer dell'utente, non può essere usato nel get. Deve contenere l'attributo **enctype="multipart/form-data"**, che vuol dire che questo form spedisce diversi tipi di dati sotto forma di coppia chiave-valore, sotto forma di dati in formato binario.

Tag <textarea>

Permette di inviare un testo di più righe. Sono obbligatori gli attributi **rows** e **cols** per aiutare il browser a renderizzare più in fretta. Ha un tag di apertura e uno di chiusura.

Tag <select>

Permette di creare menù a tendina. Non sono il massimo per l'accessibilità perché serve una certa precisione del mouse, anche se alcuni browser mitigano questo problema. Se si può evitare, soprattutto se con elenchi lunghi, meglio.

Tag <datalist>

Come i menù a tendina ma posso scrivere per ottenere i valori che corrispondono.

Innovazioni

Innovazioni per le form

- Al tag **<form>** vengono aggiunti i seguenti attributi
 - **target**: indica dove visualizzare la risposta (**_blank**, **_self**, **_parent**, **_top**, **_iframename**)
 - **autocomplete**, va usato
 - **novalidate**, serve per non applicare la validazione di XHTML5
- E i seguenti tag:
 - **keygen** per generare le chiavi per un sistema crittografico

- **menu** per i menù contestuali
- **output** che funge da segnaposto per i risultati di un calcolo

Innovazioni più interessanti: - Al tag `<input>` vengono aggiunti i seguenti valori per l'attributo **type**, aggiunti soprattutto per i siti visualizzati da cellulare. Vengono visualizzati in modo diverso, e quindi permettono un input diverso. Inoltre offrono già una validazione dell'input. - **number** (inserisce due freccette per aumentare o diminuire il valore, ma rimane editabile), **range** - **color**, si apre un color-picker - **email**, controlla che la mail sia sinteticamente corretta - **url** - **tel** - **search**, casella di ricerca - **datetime**, **datetime-local**, **date**, **month**, **time**, **week**, aprono un calendario

Nuovi attributi per i controlli - **required** - **formnovalidate** - **pattern**: contiene un'espressione regolare per la validazione dell'input, serve ad aggiungere dei vincoli - **placeholder**: contiene un suggerimento - **autocomplete**, **autofocus** - **spellcheck** - **min**, **max**, **step** - **multiple**, permette di selezionare più file

Attributo **autocomplete** - Aiuta a velocizzare le operazioni di completamento di una form - **name**: nome complete - **given-name**: nome - **family-name**: cognome - Non richiedere il titolo (dott., prof. ecc.) - Utilizzare **spellcheck="false"**

Tag "nocivi" (P. Griffiths) - Sono tag che si occupano di aspetti di presentazione o tag non validi - Presentazionali: `<b>`, `<i>`, `<big>`, `<small>`, `<marquee>`, `<blink>`, `<u>`, `<tt>`, `<sub>`, `<sup>`, `<center>`, `<hr>`, etc. - Altri: `<applet>` e `<embed>` (si deve usare `<object>`), `<font>`, `<frame>`, `<frameset>`, `<iframe>`, etc. - ATTENZIONE: HTML5 permette l'uso di `iframe` e di `small`

Canvas

Permette con javascript di disegnare delle forme, o animazioni, con grafica vettoriale (quindi molto leggere da processare) in modo da inserirle all'interno del sito. Deve essere usato solo quando appropriato (quindi non per disegnare un header). È necessario impostare dei fallback

```
<canvas id="canvasID" width="300" height="200">
</canvas>
```

Cosa si può fare con Canvas? - Disegnare forme, testo, linee e curve - Colorare forme, testo, linee e curve - Creare gradienti e pattern - Copiare immagini, immagini di un video e altri canvas - Manipolare i pixel - Esportare il contenuto di un canvas in un file statico

Tutto ciò che viene fatto nelle canvas viene realizzato tramite JavaScript. - Canvas 2D API - <http://html5doctor.com/an-introduction-to-the-canvas-2d-api/>

```
var canvas = document.getElementById('canvasID');
var context = canvas.getContext('2d');
context.strokeStyle = '#990000';
context.strokeRect(20,30,100,50);
```

```
strokeRect(left, top, width, height);
fillStyle, fillRect, lineWidth, shadowColor, ...
```

Bisogna stare attenti a livello di accessibilità perché tutto quello che faccio con canvas devo poi preoccuparmi io di descriverlo per le tecnologie assistive.

Audio

HTML5 permette di supportare la riproduzione di file audio in modo nativo. Esempio di come NON va usato:

```
<audio src="song.mp3" autoplay loop controls />
```

L'autoplay, in generale, non è una bella idea. Controls sono i controlli per l'audio (pausa, velocizza ecc.).

Altro modo per farlo:

```
<audio controls="controls">
  <source src="song.ogg" type="audio/ogg" /> <!-- PRIMA PROVA QUESTO-->
  <source src="song.mp3" type="audio/mp3" /> <!-- POI PROVA QUESTO-->
</audio>
<p>Testo sostitutivo dell'audio.</p> <!-- INFINE PROVA QUESTO-->
```

Il tag <p> è stato messo fuori perché se il browser è troppo vecchio, e non riconosce il tag <audio>, cancella tutto quello che c'è al suo interno. Quindi, per evitare che il testo sostitutivo venga cancellato, lo si mette fuori e poi con css si mette una media query per vedere se è in grado di riprodurre l'audio. Stiamo parlando però di browser vecchi di almeno 10 anni. In base al target quindi va bene anche metterlo dentro. Se lo si mette dentro il rendering è più veloce.

Questo viene fatto nel caso il browser non riesca a riprodurre alcuni formati. Funziona allo stesso modo anche con i video.

Video

Funziona in modo simile a <audio>, con l'aggiunta di un po' di attributi.

```
<video width="320" height="240" controls="controls" poster="img.jpg">
  <source src="movie.mp4" type="video/mp4" />
  <source src="movie.ogv" type="video/ogg" />
  <p>Testo alternativo al video.</p>
</video>
```

Siccome <audio> e <video> sono dei tag, posso farci tutto quello che faccio con gli altri tag (dargli un css o un comportamento). Attenzione che già la riproduzione di un video costa risorse, se gli si aggiungono troppi effetti di stile si richiedono ancora più risorse.

Lavorare in locale

Queste sono tutte cose introdotte non tanto per lo sviluppo web quanto per le web application.

Per salvare i dati di un client prima si potevano utilizzare i cookies, ma questi erano molto limitati. HTML5 offre tre diverse alternative: Web Storage, Web SQL Database e IndexedDB - Web Storage offre due oggetti sessionStorage e localStorage che memorizzano i dati sotto forma di coppie <nome, valore>. sessionStorage viene eliminato quando l'utente chiude la pagina (interrompe la sessione). localStorage rimane finché l'utente non va a posta ad eliminare la cache.

È il più utilizzato. - Web SQL Database è un database relazionale - IndexedDB si basa su una memorizzazione basata su oggetti indicizzati. Molto veloce ed efficiente.

Cache manifest

La crescente diffusione dei dispositivi mobili richiede la necessità di sviluppare applicazioni che possono lavorare offline, ovvero senza un costante collegamento alla rete. - es. Gmail, Calendar Il download delle risorse che saranno disponibili anche in assenza di rete avviene in modo trasparente all'utente.

Il file .manifest, chiamato anche cache manifest, elenca le risorse disponibili anche in assenza di connessione alla rete - La prima riga deve contenere la stringa **CACHE MANIFEST** - I commenti si esprimono con # e devono apparire su una riga a parte - Il file è organizzato in sezioni: - **CACHE:**, è elenco di risorse che sono disponibili anche quando non c'è rete. Quando entro nella home di un sito che ha la cache manifest il browser va a guardarselo e fa il download di tutte le pagine che sono nella sezione cache. - **NETWORK:**, tutte le pagine che NON vanno scaricate perché possono essere utilizzate solo in presenza di rete - **FALLBACK:**, ha un entry per ogni pagina in NETWORK che dice la pagina da fornire se la rete non c'è. È importante che dica che si tratta di problema di rete, se no l'utente capisce che è il sito che non va.

```
CACHE MANIFEST
# versione 0.1
```

CACHE:

Risorsa1.html

Risorsa2.html

NETWORK:

Aggiorna.cgi

FALLBACK:

Online.html Offline.html

/news/* avviso.html

Esempio Risorsa1.html

```
<!DOCTYPE html>
```

```
<html manifest="risorsa.manifest" >
```

```
...
```

```
</html>
```

Il file che contiene il riferimento al file .manifest viene comunque conservato in locale anche se non presente nel file .manifest. Il file .manifest deve essere servito con il tipo MIME corretto, ovvero text/cache-manifest.